

CURA

Enterprise System Architecture & Software Engineering Blueprint

Official Standard: Clean Architecture • SOLID Principles • Multi-Team Contracts

OFFICIAL ENGINEERING TEAM DISTRIBUTION (11 ENGINEERS)

1. AI & BACKEND CORE
Lead Architect (1 Person)
Edge AI on Pi 4 (Qwen 2.5, Whisper, Piper), FastAPI Orchestrator, Tool Calling Router.

2. DESKTOP GUI
2 Engineers
PyQt6/Electron/Flutter Desktop, MVVM Pattern, Adaptive UI Modes (Normal, Alzheimer's, ADHD).

3. MOBILE APPLICATION
2 Engineers
Flutter/React Native, Patient & Caregiver/Doctor Views, FCM Push Alerts, Geofencing SOS.

4. WEB FRONTEND
2 Engineers
Next.js 14+ / React + TS, Public Showcase, "Try CURA" Voice Demo, Caregiver Web Portal.

5. EMBEDDED HARDWARE
3 Engineers
Raspberry Pi 4 (4GB), Mic Array, Speaker, 4" LCD, WS2812B NeoPixel Ring, Hardware Emergency Button (No Camera).

6. DATABASE ENGINEERING
1 Engineer
PostgreSQL 3NF Schema, Redis Caching, Audit Logs, Edge SQLite Sync, Row-Level Security.

Official Project Name: CURA (Strictly CURA)

Target Edge SBC: Raspberry Pi 4 Model B (4GB RAM)

Date: October 2026 • Approved for Implementation

EXECUTIVE SUMMARY

1. Architectural Vision & System Principles

CURA is an adaptive, multi-modal cognitive assistant engineered to support neurodivergent individuals (ADHD) and memory-impaired seniors (Alzheimer's disease), alongside high-efficiency everyday voice automation. The system dynamically alters its personality, UI complexity, interaction pacing, and sensory output based on active cognitive profiles.

Key Architecture Directives for All Teams:

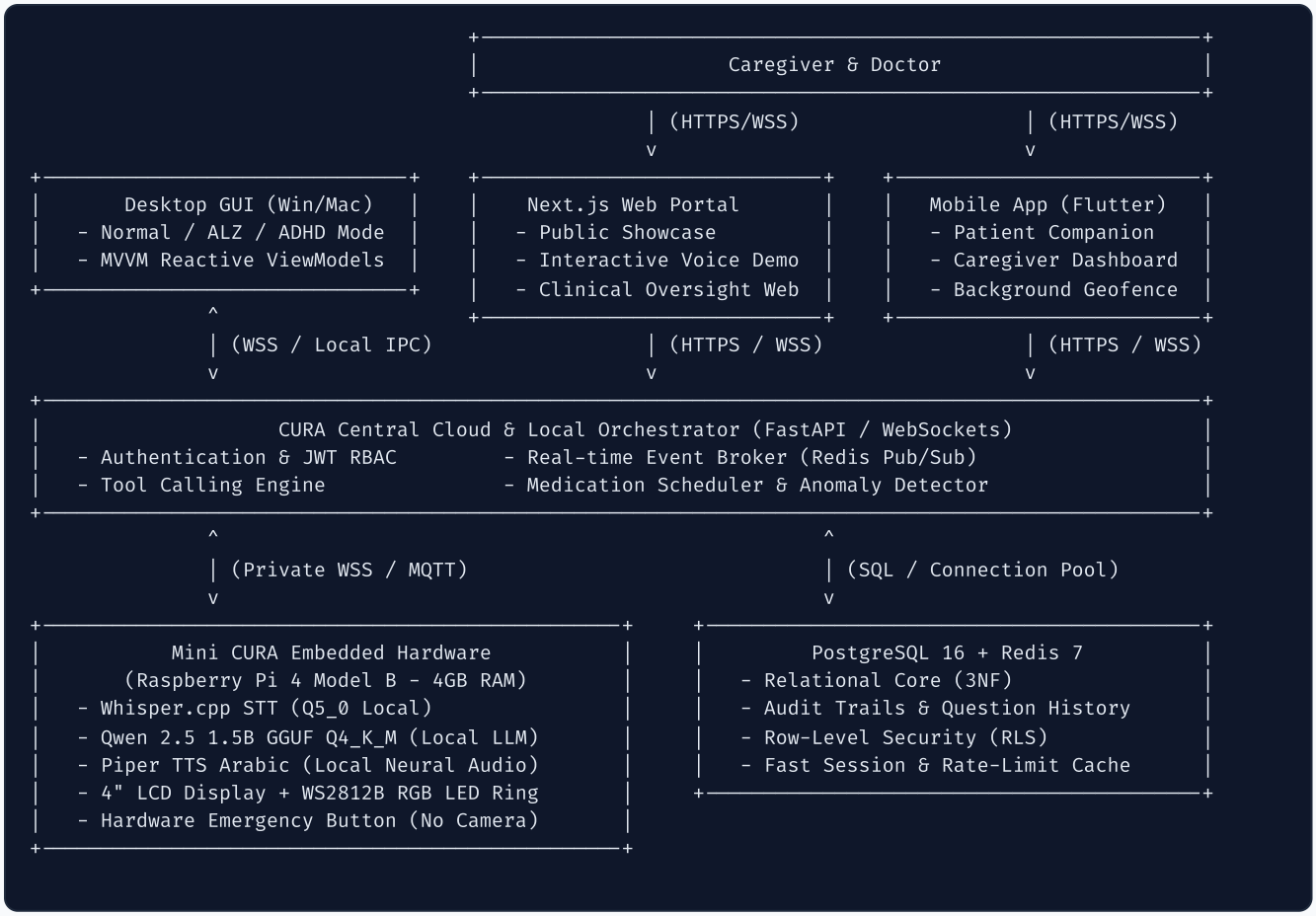
- **Official Naming Policy:**
  The official name of the project is
  CURA
  and nothing else. Legacy references to JAS or JARVIS in historical drafts are superseded.
- **Edge AI Hardware Isolation:**
  The machine learning pipeline (STT, LLM, TTS) is deployed directly on a
  Raspberry Pi 4 Model B (4GB RAM)
  . The camera module is explicitly excluded to preserve 100% of CPU and RAM for sub-second voice and language inference.
- **Network Connectivity:**
  The Raspberry Pi hardware operates connected via
  Wi-Fi 802.11ac
  to cloud services and local client GUIs over secure WebSockets and REST APIs.
- **Total Memory Envelope:**
  The on-device memory consumption must not exceed
  2.2 GB – 2.8 GB RAM
  under full load, leaving headroom for the OS and preventing Out-Of-Memory (OOM) fatal kills.

Core Software Engineering Principles Enforced Across All Teams

Principle	Software Engineering Rule	Application in CURA
S — Single Responsibility (SRP)	A class, module, or service must have one and only one reason to change.	Separate audio capture, transcription, prompt construction, tool execution, and voice synthesis into discrete decoupled classes.
O — Open/Closed (OCP)	Software entities should be open for extension, but closed for modification.	Add new cognitive modes or external tool-calls by creating new polymorphic strategy classes without touching core router code.
L — Liskov Substitution (LSP)	Subtypes must be substitutable for their base types without altering correctness.	All audio sinks (Speaker, Virtual Stream, Test Mock) implement `IAudioOutput` interchangeably.
I — Interface Segregation (ISP)	Clients should never be forced to depend on interfaces they do not use.	Desktop GUI subscribes to `IStateObserver`, ignoring raw audio processing interfaces.
D — Dependency Inversion (DIP)	High-level modules must not depend on low-level modules. Both must depend on abstractions.	AI Orchestrator injects `ILLMEngine` rather than depending concretely on `llama.cpp` or `Ollama`.
Clean Architecture	Independent of frameworks, UI, databases, and external agencies.	Domain entities and business use cases remain pure without framework imports.
Contract-First Development	Define OpenAPI 3.1 & AsyncAPI specifications before writing implementation code.	Ensures Mobile, GUI, Web, Backend, and Hardware teams work in parallel against mock servers.

Global Macro-Architecture (C4 Container View)

FIGURE 1.1: CURA END-TO-END ENTERPRISE SYSTEM TOPOLOGY



TEAM 1 • 1 ENGINEER (LEAD ARCHITECT - ME)

2. AI Pipeline & Backend Core Architecture

As the architect and backend lead, your role is the backbone of CURA: you govern the on-device AI pipeline on the Raspberry Pi 4 (4GB RAM), tool execution, and the central API service orchestrating hardware and multi-platform clients.

2.1 On-Device Local AI Specifications (Raspberry Pi 4)

Pipeline Stage	Engine & Architecture	Model / Quantization	RAM Footprint	Inference Latency & Tuning
1. Audio Input & VAD	Silero VAD / WebRTC VAD + PyAudio	C++ Optimized Binary	~30 MB	Continuous background buffer, <10ms detection latency.
2. STT (Speech-to-Text)	Whisper.cpp (FR-NJ-001, FR-HW-001)	base or small (Q5_0 GGUF)	~180 – 240 MB	Runs on 4 CPU threads with NEON instructions. Word Error Rate < 9% on Egyptian Arabic.
3. LLM Reasoning	llama.cpp / Ollama Server	Qwen 2.5 1.5B-Instruct (Q4_K_M)	~1.05 GB	**8–10 tokens/sec** on Pi 4 CPU. Native Arabic grammar + Tool Calling capability.
4. TTS (Speech Synthesis)	Piper TTS (ONNX Runtime)	Arabic Voice Model (ar_EG)	~65 MB	Sub-second synthesis: **< 0.2s latency**. Direct streaming to ALSA audio sink.
5. Python Runtime	FastAPI + asyncio + uvloop	Python 3.11 64-bit	~180 MB	Coordinates hardware I/O, event listeners, and API sync.
Operating System	Raspberry Pi OS Lite (64-bit)	Debian Bookworm Kernel	~450 MB	Headless, zero X11/Wayland overhead.
Total System Memory Ceiling (Target: ≤ 2.8 GB)			**~2.15 – 2.45 GB**	**Safe Headroom: ~1.5 GB available RAM.**

2.2 Clean Architecture Design for AI Core

```
cura_ai_backend/
├── src/
│   ├── core/                    # Domain Core (Zero external framework imports)
│   │   ├── entities/            # CognitiveMode, Intent, VoiceCommand, MedicationPlan
│   │   ├── interfaces/          # ISTTEngine, ILLMEngine, ITTSEngine, IToolRouter
│   │   └── use_cases/           # ProcessVoiceInteraction, HandleEmergencyAlert, SummarizeCaregiverLog
│   ├── infrastructure/          # Low-Level Concrete Adapters
│   │   ├── speech/              # WhisperCppAdapter, PiperTTSAdapter
│   │   ├── llm/                 # LlamaCppClient, OllamaClient, QwenPromptBuilder
│   │   ├── tools/               # DuckDuckGoTool, WeatherTool, MedicalHistoryTool
│   │   └── database/            # PostgresRepository, RedisClient
│   └── presentation/            # Interfaces to external world
│       ├── api/                 # FastAPI Routers (REST endpoints)
│       └── websockets/          # WebSocket Voice Stream & Hardware State Bridge
```

2.3 Production Tool Calling Router Implementation

```
# src/core/use_cases/tool_calling_router.py
from typing import Dict, Any, Callable
import json

class ToolRouter:
    """Open/Closed Principle: Register tools without mutating the core loop."""
    def __init__(self):
        self._registry: Dict[str, Callable] = {}

    def register_tool(self, name: str, handler: Callable):
        self._registry[name] = handler

    async def execute(self, tool_name: str, arguments: Dict[str, Any]) → str:
        if tool_name not in self._registry:
            return json.dumps({"error": f"Tool '{tool_name}' not registered."})
        try:
            result = await self._registry[tool_name](**arguments)
            return json.dumps({"status": "success", "data": result}, ensure_ascii=False)
        except Exception as e:
            return json.dumps({"status": "failure", "error": str(e)})
```

Lead Developer Deliverables (Milestones 1 – 4):

1. **Milestone 1 (Laptop Prototyping):**
   Benchmark `Whisper.cpp` + `Qwen 2.5 1.5B Q4_K_M` + `Piper TTS` on laptop using 2GB VRAM or CPU. Verify zero-lag conversation.
2. **Milestone 2 (Tool Calling):**
   Implement tool calling for live weather, web search, 2-day caregiver summary (`FR-ALZ-009`), and voice medication confirmation (`FR-ALZ-007`).
3. **Milestone 3 (Local AI Server):**
   Build WebSocket state publisher synchronizing hardware LED states (Blue, Green, Yellow, Red) and screen payloads.
4. **Milestone 4 (Pi 4 Deployment):**
   Flash SanDisk Extreme A2 on Raspberry Pi 4 (4GB), profile RAM under stress tests, and verify continuous thermal stability with active fan.

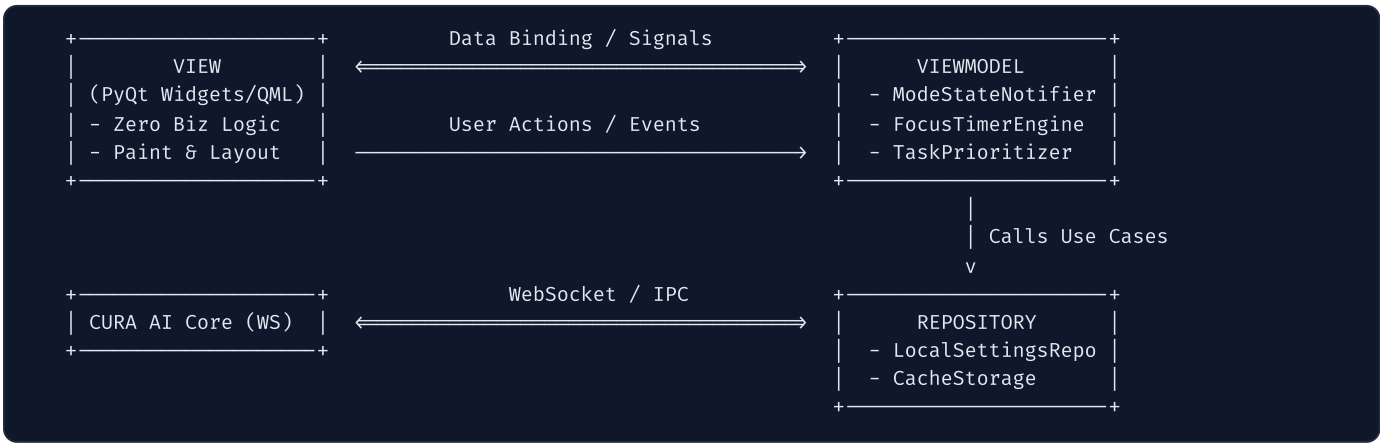
TEAM 2 • 2 ENGINEERS

3. Desktop GUI Architecture (PyQt6 / Electron / Flutter)

The Desktop GUI team delivers the primary workstation user interface. It provides low-friction ambient assistance during work hours (Normal Mode), executive function scaffolding (ADHD Mode), and simplified, high-contrast clarity for memory care (Alzheimer's Mode).

3.1 Architecture Pattern: Model-View-ViewModel (MVVM)

To prevent "Massive View Controllers" and tightly coupled widget bugs, the team must strictly isolate UI widgets from data processing:



3.2 Three Adaptive Modes Implementation Specifications

Mode	Visual Styling & Palette	Key Components & Widgets	Anti-Failure Rule
Normal CURA	Dark Midnight Obsidian (`#131B26`) with Electric Cyan glow (`#60EFEB`).	Floating Ambient Orb, Waveform Audio Visualizer, Quick Command Palette (Cmd+K), Screen Awareness Inspector.	Zero audio lag during speech capture; never block UI thread on network calls.
Alzheimer's Mode	High-contrast, large elements. Slate background with Mint confirmations (`#3FEDBA`).	Giant Persistent Date/Time/Day banner (`FR-ALZ-005`), Medication cards with photo verification, SOS Emergency button, Memory Book carousel.	**Zero nested sub-menus.** All primary actions reachable in 1 click. Font size minimum 18pt.
ADHD Focus Mode	Calm Obsidian with Orchid Violet sprint accents (`#AF6AFB`). Minimalist.	Single Task Focus Card (`FR-ADHD-004`), Sprint Progress Bar, Impulsive Action Pause-and-Confirm Dialog (`FR-ADHD-001`), Guided Reset Breathing Modal.	**Never show task backlogs** during active sprint. Hide all non-essential badges and alerts.

3.3 Clean Code Standard: Reactive ViewModel

```
# presentation/gui/viewmodels/adhd_focus_viewmodel.py
from PyQt6.QtCore import QObject, pyqtSignal

class AdhdFocusViewModel(QObject):
    task_changed = pyqtSignal(str)
    progress_updated = pyqtSignal(float)
    impulse_alert_triggered = pyqtSignal(str)

    def __init__(self, task_service, audio_feedback):
        super().__init__()
        self._task_service = task_service
        self._audio = audio_feedback
        self._current_task = None

    def attempt_switch_app(self, target_app: str):
        # Rule FR-ADHD-001: Intercept impulsive application switching
        if self._task_service.is_focus_mode_active():
            self.impulse_alert_triggered.emit(
                f"You are in focus sprint! Proceed with opening {target_app}?"
            )
            self._audio.play_gentle_pause_tone()
        else:
            self._task_service.launch_app(target_app)
```

GUI Team Mandatory Mandates:

- All long-running tasks (audio streaming, API requests, heavy calculations) **must** execute in worker threads (`QThread` or async tasks). Freezing the GUI event loop for even 50ms is unacceptable.
- Implement full keyboard accessibility and screen-reader semantics for all Alzheimer's controls.

TEAM 3 • 2 ENGINEERS

4. Mobile Application Architecture (Flutter / React Native)

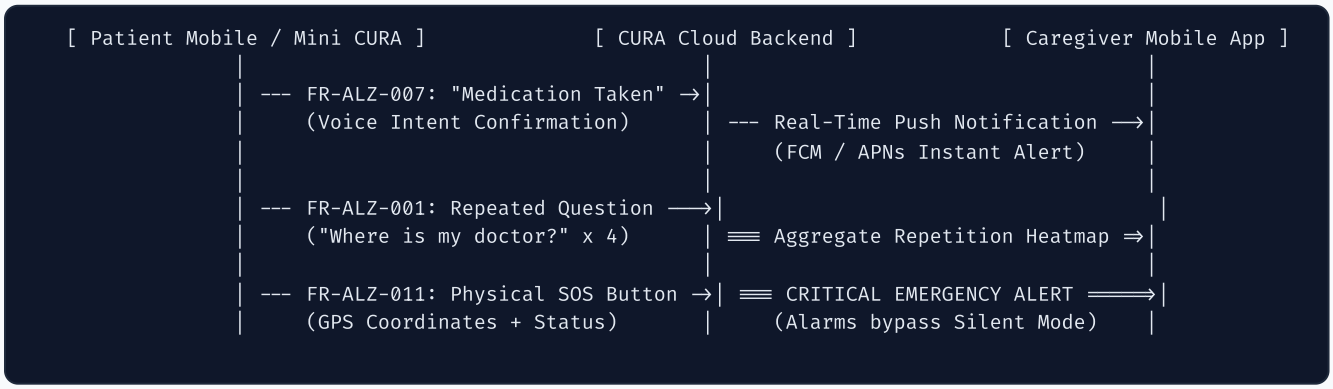
The Mobile team connects users anywhere on iOS and Android. It delivers dual experiences: an intuitive companion for the patient and a real-time clinical monitoring dashboard for caregivers and physicians.

4.1 Clean Architecture Layered Directory Structure

```
lib/
├── core/
│   ├── network/                  # Dio/Http client with JWT interceptors & auto-refresh
│   ├── theme/                    # CURA Design Tokens (Colors, Gradients, Typography)
│   └── errors/                   # Failure & Exception classes
├── features/
│   ├── alzheimers_companion/     # Patient View: Simplified Reminders, Big SOS Button
│   │   ├── domain/               # Entities & Use Cases
│   │   ├── data/                 # Models & DataSources (Remote API + Local Hive/Isar)
│   │   └── presentation/         # BLoC / Riverpod StateNotifier + High-Contrast UI
│   ├── adhd_companion/           # Patient View: One Task at a time, Pomodoro Timer
│   ├── caregiver_dashboard/      # Caregiver View: 2-Day Analytics, Geofence Map, Adherence Log
│   └── emergency/                # Instant Background SOS & GPS Coordinate Broadcaster
```

4.2 Caregiver & Doctor Real-Time Monitoring Subsystem

FIGURE 4.1: CAREGIVER & PATIENT SYNCHRONIZATION FLOW



4.3 Mobile Contract: Repository Pattern Implementation

```
// features/medication/domain/repositories/i_medication_repository.dart
abstract class IMedicationRepository {
  Future<Either<Failure, List<MedicationDose>>> getTodaySchedule();
  Future<Either<Failure, void>> confirmDoseTaken(String doseId, String voiceTranscription);
  Stream<EmergencyAlert> subscribeToEmergencyEvents(String patientId);
}

// features/medication/data/repositories/medication_repository_impl.dart
class MedicationRepositoryImpl implements IMedicationRepository {
  final MedicationRemoteDataSource remoteSource;
  final MedicationLocalDataSource localCache;
  final NetworkInfo networkInfo;

  MedicationRepositoryImpl({required this.remoteSource, required this.localCache, required this.networkInfo});

  @override
  Future<Either<Failure, void>> confirmDoseTaken(String doseId, String transcription) async {
    // Offline-first: write to local database immediately, queue background sync if offline
    await localCache.markDoseConfirmed(doseId);
    if (await networkInfo.isConnected) {
      return await remoteSource.sendDoseConfirmation(doseId, transcription);
    } else {
      await localCache.queuePendingSync(doseId);
      return const Right(null);
    }
  }
}
```

Mobile Team Mandates:

- Emergency alert push notifications **must** have Critical Alert entitlement (bypassing phone Do Not Disturb mode for authorized caregivers).
- Background location services for wandering Alzheimer's patients (`FR-ALZ-004`) must use geofencing to preserve mobile battery life.

TEAM 4 • 2 ENGINEERS

5. Web Frontend Architecture (Next.js 14+ / React / TS)

The Web Frontend team builds the public gateway and clinical portal for CURA. Built with Next.js 14+ (App Router), it delivers an SEO-optimized showcase landing page, an interactive browser demo ("Try CURA"), and a desktop-optimized Caregiver & Physician portal.

5.1 Feature-Sliced Directory Architecture

```
src/
├── app/                          # Next.js App Router (Layouts, Pages, Route Handlers)
│   ├── (marketing)/              # Public Route Group: Home, Overview, Features, Team
│   │   ├── page.tsx              # FR-WEB-001: Cinematic Landing Page
│   │   ├── features/page.tsx     # FR-WEB-003: Interactive Mode Feature Comparison
│   │   ├── team/page.tsx         # FR-WEB-004: Engineering & Project Team Showcase
│   │   └── try-cura/page.tsx     # FR-WEB-005: "Try CURA" Web Audio Voice Demo
│   └── (portal)/                 # Protected Route Group: Doctor / Caregiver Portal
│       └── dashboard/            # Clinical analytics, cognitive trends, medication adherence
├── components/                   # Atomic Component System
│   ├── ui/                       # Atoms: Button, Card, Badge, Slider, Waveform
│   ├── modules/                  # Organisms: Header, VoiceVisualizer, AdherenceChart
│   └── templates/                # Page Shells
├── lib/                          # Utility functions, API Client, Zod Validation Schemas
└── styles/                       # Master CSS Tokens (Vanilla CSS / Tailwind config)
```

5.2 "Try CURA" Interactive Voice Demo Architecture (`FR-WEB-005`)

To let reviewers and visitors experience CURA directly from any modern browser without installing local models, the web demo implements Web Audio API capturing audio chunks and streaming them to the backend orchestrator:

```tsx
// components/modules/VoiceDemoRecorder.tsx
'use client';
import React, { useState, useRef } from 'react';

export const VoiceDemoRecorder: React.FC = () => {
  const [isRecording, setIsRecording] = useState(false);
  const [responseAudio, setResponseAudio] = useState<string | null>(null);
  const mediaRecorderRef = useRef<MediaRecorder | null>(null);

  const startVoiceSession = async () => {
    const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
    const mediaRecorder = new MediaRecorder(stream, { mimeType: 'audio/webm' });
    const audioChunks: Blob[] = [];

    mediaRecorder.ondataavailable = (event) => audioChunks.push(event.data);
    mediaRecorder.onstop = async () => {
      const audioBlob = new Blob(audioChunks, { type: 'audio/webm' });
      const formData = new FormData();
      formData.append('audio_file', audioBlob, 'voice_input.webm');

      const response = await fetch('/api/v1/voice/demo-interact', {
        method: 'POST',
        body: formData,
      });
      const result = await response.json();
      setResponseAudio(result.audio_url);
    };

    mediaRecorder.start();
    setIsRecording(true);
    mediaRecorderRef.current = mediaRecorder;
  };

  return (
    <div className="cura-recorder-card">
      <button onClick={isRecording ? () => mediaRecorderRef.current?.stop() : startVoiceSession}>
        {isRecording ? "Listening... (Click to Stop)" : "Speak to CURA"}
      </button>
    </div>
  );
};
```

Web Frontend Performance Targets:

- **Core Web Vitals:**
  Largest Contentful Paint (LCP) < 1.2s; Cumulative Layout Shift (CLS) = 0.
- **Responsive Accessibility:**
  100% WCAG 2.1 AA compliance with full dark/light palette mode support.

TEAM 5 • 3 ENGINEERS

6. Embedded Hardware Engineering Architecture

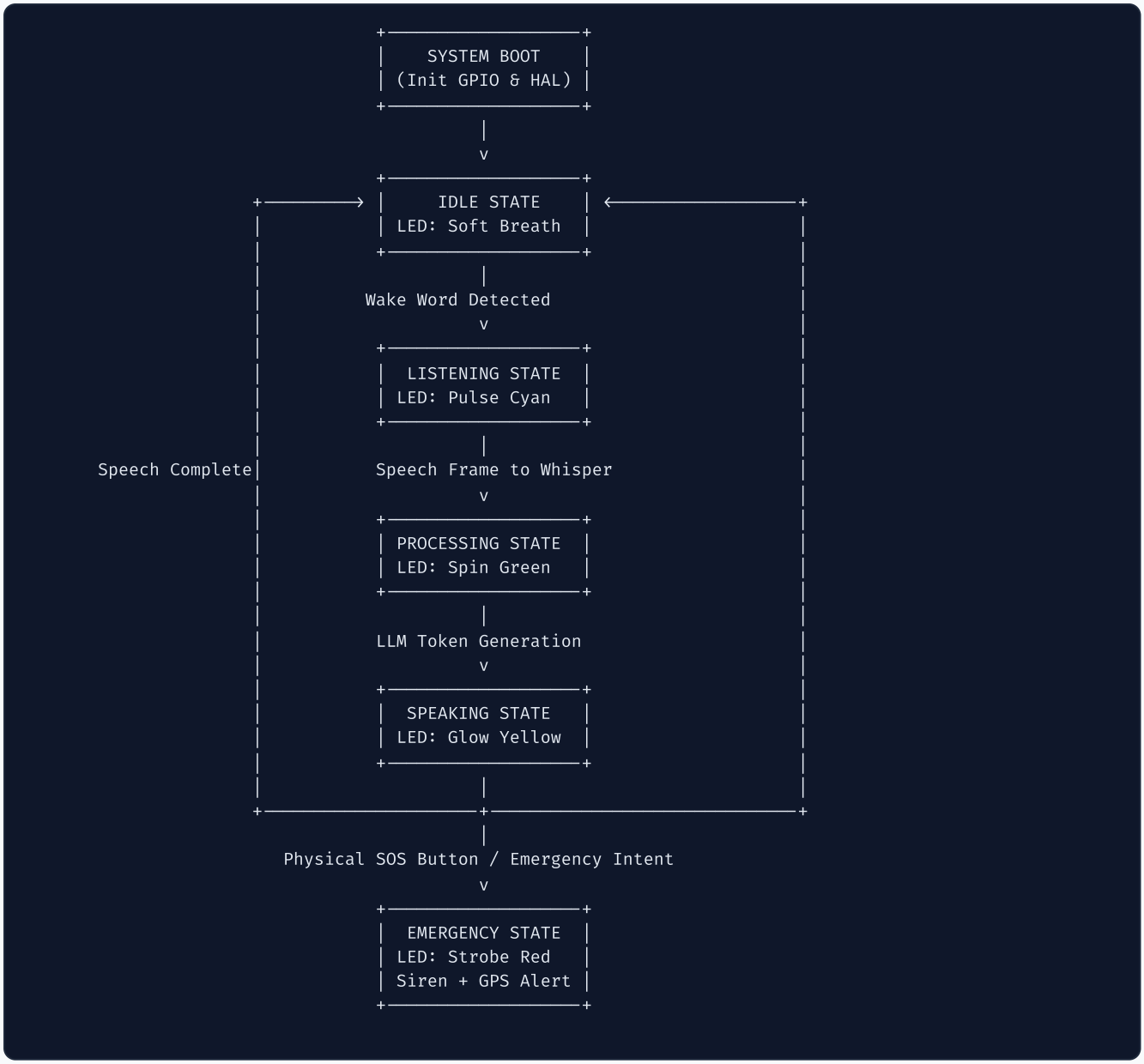
The Hardware team designs and builds the physical **Mini CURA** desktop companion. Driven by a **Raspberry Pi 4 Model B (4GB RAM)**, it executes on-device intelligence without bulky accessories or expensive custom silicon.

6.1 Hardware Bill of Materials (BOM) & Pinout Allocation

Component	Model / Part	Interface / Bus	Raspberry Pi 4 Pins	Functional Responsibility
Core SBC	Raspberry Pi 4 Model B (4GB RAM)	SoC Broadcom BCM2711	40-pin Header	Executes Whisper.cpp, Qwen 2.5 1.5B, Piper TTS, and Hardware Daemon.
Active Cooling	Official Pi Active Cooler / 5V Fan	5V GPIO / GND	Pin 4 (5V), Pin 6 (GND)	Prevents thermal throttling; keeps SoC < 65°C during heavy LLM generation.
Audio Input	Far-Field USB Mic Array / ReSpeaker	USB 2.0 / I2S	USB Port / GPIO 18, 19, 20, 21	Beamforming and hardware noise cancellation for Egyptian Arabic speech.
Audio Output	I2S DAC Amp (MAX98357A) + 3W 4Ω Speaker	I2S Digital Audio	GPIO 18 (BCLK), 19 (LRC), 21 (DIN)	Crystal-clear, zero-static voice synthesis playback via Piper TTS.
RGB LED Ring	16× WS2812B NeoPixel Ring	PWM / SPI Data	GPIO 18 (PWM0) or GPIO 10 (SPI)	FR-HW-006 Status Ring: Blue (Listening), Green (Processing), Yellow (Speaking), Red (SOS).
Status Display	4.0" TFT LCD (800×480) with Capacitive Touch	DSI / HDMI / SPI	DSI Ribbon or HDMI Port	FR-HW-007 Display: Persistent clock, medication pill cards, focus sprint timer.
Emergency Button	Tactile Push Button (Industrial Grade)	Hardware Interrupt	GPIO 17 (Pin 11) + 10kΩ Pull-up	FR-HW-004 Instant SOS trigger. Zero-latency interrupt; works even if Wi-Fi drops.
High-Speed Storage	SanDisk Extreme A2 MicroSD (64GB)	UHS-I / SDIO	MicroSD Slot	High random read IOPS (4000 IOPS) ensuring rapid GGUF model memory-mapping.

6.2 Embedded Finite State Machine (FSM)

FIGURE 6.1: MINI CURA HARDWARE STATE MACHINE



6.3 Hardware Daemon & Debounced Interrupt Handler

```
# hardware/daemon/emergency_listener.py
import RPi.GPIO as GPIO
import time
import asyncio
import aiohttp

BUTTON_PIN = 17
DEBOUNCE_MS = 250

class EmergencyTriggerDaemon:
    def __init__(self, api_endpoint: str):
        self.api_endpoint = api_endpoint
        self.last_press_time = 0
        GPIO.setmode(GPIO.BCM)
        GPIO.setup(BUTTON_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)
        GPIO.add_event_detect(BUTTON_PIN, GPIO.FALLING, callback=self._handle_interrupt,
bouncetime=DEBOUNCE_MS)

    def _handle_interrupt(self, channel):
        current_time = time.time() * 1000
        if current_time - self.last_press_time > DEBOUNCE_MS:
            self.last_press_time = current_time
            print("[HARDWARE INTERRUPT] Emergency Push Button Pressed!")
            asyncio.run(self._broadcast_emergency())

    async def _broadcast_emergency(self):
        async with aiohttp.ClientSession() as session:
            payload = {"source": "mini_cura_hardware_button", "timestamp": time.time()}
            await session.post(f"{self.api_endpoint}/api/v1/emergency/trigger", json=payload)
```

Hardware Engineering Directives:

- **No Camera Confirmation:**
  Camera module is completely excluded. Resources are 100% focused on audio quality, display smoothness, and zero-drop Wi-Fi communication.
- **Systemd Daemon Supervision:**
  Hardware service runs under systemd with `Restart=always` and `RestartSec=3s`.

TEAM 6 • 1 ENGINEER

7. Database Architecture & Data Governance

The Database engineer guarantees transactional consistency, data privacy, and sub-10ms query response times. The architecture employs a hybrid strategy: **PostgreSQL 16** for relational data, **Redis 7** for real-time pub/sub caching, and **SQLite** on edge devices.

7.1 Complete Entity-Relationship (ER) Schema

Table Name	Primary Key	Foreign Keys	Key Attributes & Indexes	Integrity Rules
`users`	`id (UUID)`	—	`email`, `password_hash`, `role (Enum: patient, caregiver, doctor)`, `phone`	Unique email; B-tree index on email.
`patient_profiles`	`id (UUID)`	`user_id → users(id)`	`active_mode (Enum: normal, alzheimer, adhd)`, `emergency_contact_phone`, `date_of_birth`	Cascade on delete; 1–to-1 with users.
`caregiver_assignments`	`id (UUID)`	`caregiver_id → users(id)` `patient_id → patient_profiles(id)`	`relationship`, `access_level`, `created_at`	Unique pair (caregiver, patient).
`medications`	`id (UUID)`	`patient_id → patient_profiles(id)`	`name`, `dosage`, `instructions`, `photo_url`, `schedule_cron`	Indexed on patient_id.
`medication_adherence_logs`	`id (UUID)`	`medication_id → medications(id)`	`scheduled_time`, `confirmed_at`, `status (Enum: taken, missed, pending)`, `voice_transcript`	Time-series index on scheduled_time.
`question_repetition_logs`	`id (UUID)`	`patient_id → patient_profiles(id)`	`question_text`, `normalized_intent`, `timestamp`, `repetition_count_last_24h`	Indexed on (patient_id, timestamp).
`tasks`	`id (UUID)`	`patient_id → patient_profiles(id)`	`title`, `urgency (1-5)`, `priority (1-5)`, `estimated_minutes`, `is_completed`	Used by ADHD Prioritization engine.
`focus_sessions`	`id (UUID)`	`task_id → tasks(id)`	`start_time`, `duration_minutes`, `interruptions_count`, `was_completed`	ADHD progress streak analytics.
`emergency_events`	`id (UUID)`	`patient_id → patient_profiles(id)`	`trigger_source (button, voice)`, `latitude`, `longitude`, `resolved_at`	Immutable audit log; critical priority.

7.2 Row-Level Security (RLS) Policy Example

```
-- Enable RLS on Sensitive Patient Health Records
ALTER TABLE medication_adherence_logs ENABLE ROW LEVEL SECURITY;

-- Caregivers can only read medication logs of patients assigned to them
CREATE POLICY caregiver_patient_isolation ON medication_adherence_logs
    FOR SELECT
    USING (
        EXISTS (
            SELECT 1 FROM caregiver_assignments ca
            JOIN patient_profiles pp ON pp.id = ca.patient_id
            JOIN medications m ON m.patient_id = pp.id
            WHERE ca.caregiver_id = auth.uid()
            AND m.id = medication_adherence_logs.medication_id
        )
    );
```

CROSS-TEAM STANDARDS

8. Inter-Team API Contracts & Communication Protocols

All teams communicate over strictly enforced contracts. Every JSON response follows the **Universal Envelope Pattern** to eliminate ambiguous payload parsing across teams:

8.1 Standardized API Response Envelopes

SUCCESS RESPONSE ENVELOPE — 200 OK

```
{
  "status": "success",
  "data": { ... },
  "meta": {
    "timestamp": "2026-10-17T12:00:00Z",
    "version": "v1.0"
  }
}
```

ERROR RESPONSE ENVELOPE — 4XX / 5XX

```
{
  "status": "error",
  "error": {
    "code": "DOSE_ALREADY_CONFIRMED",
    "message": "Dose was verified at 10:30 AM.",
    "field": null
  }
}
```

8.2 Core API Endpoints Specification

Endpoint	Method	Consumer	Description	Key Payload / Query Parameters
`/api/v1/auth/login`	`POST`	GUI, Mobile, Web	Authenticate user and issue JWT Access + Refresh token pair.	`{ "email": "...", "password": "..." }`
`/api/v1/patients/{id}/summary`	`GET`	AI Tool, Mobile, Web	FR-ALZ-009: Fetch 2-day activity, question repetition, and medication logs.	`?days=2` (Returns structured JSON for Qwen 2.5 narration).
`/api/v1/medications/{id}/confirm`	`POST`	Mini CURA, Mobile, GUI	FR-ALZ-007: Confirm medication dose taken via voice intent or tap.	`{ "status": "taken", "confidence": 0.96, "transcript": "أخذت الدواء" }`
`/api/v1/emergency/trigger`	`POST`	Mini CURA, Mobile, GUI	FR-ALZ-011: Broadcast instantaneous SOS alert to all caregivers.	`{ "source": "hardware_button", "lat": 30.0444, "lng": 31.2357 }`
`/ws/v1/hardware/stream`	`WSS`	Mini CURA ↔ Backend	Bidirectional telemetry: LED state changes, screen cards, audio events.	Full-duplex WebSocket frame protocol.

8.3 WebSocket Real-Time Hardware Protocol (`/ws/v1/hardware/stream`)

```
// Payload sent from Backend to Mini CURA to sync LED & Screen
{
  "event": "AI_STATE_UPDATE",
  "payload": {
    "state": "SPEAKING",                // LISTENING | PROCESSING | SPEAKING | EMERGENCY | IDLE
    "led_ring": {
      "color_hex": "#F59E0B",          // Yellow
      "animation": "BREATHING_PULSE"
    },
    "screen_display": {
      "card_type": "MEDICATION_ALERT",
      "primary_text": "حان وقت دواء الضغط",
      "secondary_text": "قرص واحد بعد الغداء",
      "icon_id": "pill_blood_pressure"
    }
  }
}
```

QUALITY & DESIGN

9. Code Standards & Official CURA Brand Tokens

9.1 Testing Pyramid & Code Review Checklist

UNIT TESTING (≥ 80% COVERAGE)

Every single use case, domain entity, and repository adapter must have isolated unit tests using PyTest (Python), JUnit/MockK (Android), or Jest/Vitest (TS).

CONTRACT TESTING (PACT / OPENAPI)

Frontend, Mobile, and Hardware teams run automated contract verification against the OpenAPI JSON spec to prevent breaking payload changes.

Pull Request (PR) Merge Criteria:

- Strict static typing enabled (mypy for Python, strict TypeScript for Web, strong typing in Dart). Zero `any` types tolerated.
- Linter passes with zero warnings (ruff/black, eslint, flutter analyze).
- All new API endpoints documented in OpenAPI spec before PR review.

9.2 Official CURA Brand Color Palette & UI Tokens

All client interfaces (Desktop GUI, Mobile App, Web Frontend, and 4" Hardware Display) must strictly utilize the official CURA brand design tokens:

Azure Electric Blue `#16B9FE` Primary Brand & Core UI	**Electric Cyan** `#60EFEB` Voice Waveform & Listening	**Orchid Violet** `#AF6AFB` ADHD Sprint & Focus Mode	**Turquoise Mint** `#3FEDBA` Alzheimer Care & Confirmation
Midnight Obsidian `#131B26` Canvas Background	**Deep Navy Slate** `#1A2433` Elevated Cards & Dialogs	**Pure Studio White** `#FFFFFF` Headings & Active Icons	**Glacier Slate** `#E8EDF1` Secondary Text & Borders

9.3 CSS Gradients Ready for Web & App Implementations

```
/* CURA Signature Flow (Master Brand) */
background: linear-gradient(135deg, #16B9FE 0%, #AF6AFB 50%, #3FEDBA 100%);

/* Cognitive Focus (ADHD Mode Active Sprint) */
background: linear-gradient(135deg, #16B9FE 0%, #AF6AFB 100%);

/* Calming Care (Alzheimer's Mode Comfort Banner) */
background: linear-gradient(135deg, #16B9FE 0%, #3FEDBA 100%);
```

CURA — Adaptive AI Assistant • Graduation Project Architecture Specification • Approved for Multi-Team Development